

Torus Knot 切割与区域着色算法（现行实现）

1 总览

Torus Knot 切割把解析曲线族 $v = ku + 2\pi n$ ($n \in \mathbb{Z}$) 投影到环面网格的 UV 参数域，沿曲线把与之相交的面切开，并为每个面指派其所属的**环面拓扑区域**编号供 ByRegion 着色使用。

现行实现由两条彼此独立的链路组成，理解这一分工是全文的关键：

链路	入口	职责
几何切割	<code>cut_mesh_by_knots</code>	插顶点、加弦、标记 cut 屏障边
区域指派	<code>assign_connected_components_knot</code>	逐面按解析拓扑不变量算区域号

区域指派**不依赖**切割结果的连通性——它是纯解析公式。这一解耦是本轮重写的核心：任何基于 flood-fill 的连通块统计在环面接缝处都极其脆弱（见第 6 节），而解析公式对任意 k 、任意网格分辨率都恒定正确。

相对旧文档的实质修正

- 分支范围两端都用 $\lfloor \cdot \rfloor$ ，**不是** $n_{lo} = \lceil \phi_{min}/2\pi \rceil$ 。旧公式在 $\phi < 0$ 时算出 $n_{lo} > n_{hi}$ ，使 `for branch in n_lo..=n_hi` 退化为空循环、整面漏切（见第 3 节）。
- 切割入口是 `cut_face_knots`，一次性处理**所有曲线的所有分支**。旧的 `cut_face_local_knot` 已删除，逐曲线串行切割的“修复遍”也一并取消。
- 区域语义是**环面拓扑区域**（区域数 = $|k_2 - k_1|$ ），**不是** UV 平面条带（ $k + 1$ 条）。函数 `assign_multi_knot_patch_indices` 及其 `div_euclid` 条带向量 + BTreeSet 去重方案已删除。

2 参数域、曲线与容差

UV 域

顶点在 $u = 0$ 与 $u = 2\pi$ 处是**独立顶点**（`generate_unfolded_quad_mesh` 生成 $(res_u + 1) \times (res_v + 1)$ 顶点），没有面跨过周期边界。`clamp_uv` 把新交点夹到 $[\min, \max]$ 而不模 2π 包裹，从而保留 $u = 0$ 与 $u = 2\pi$ 为两条独立边界（接缝）。

所有涉及曲线方程的计算先经 `normalize_uv` 归一化到 $[0, 2\pi)$ ，再用 `unwrap_angle` 相对面内首顶点解缠（使 $|x - r| \leq \pi$ ），保证同一面内的顶点坐标连续、跨接缝面也不撕裂。

相位函数 ϕ

对给定 k 定义

$$\phi(u, v) = v - k u.$$

曲线的第 n 条分支即等值线 $\phi = 2\pi n$ 。分支 n 覆盖 $\phi \in [2\pi n, 2\pi(n+1))$ 。

容差常量

常量	值	用途
EPSILON	10^{-8}	分母退化判定、 t 的区间松弛
MERGE_EPS	10^{-4}	Grid 切割的“顶点在线上”判定
COINCIDENT_TOL	10^{-4}	同一面内顶点 3D 重合判定
KNOT_SNAP_UV	10^{-3}	knot 曲线顶点吸附（参数域垂直距离）

顶点吸附 KNOT_SNAP_UV 高分辨率网格下曲线经常“几乎”穿过某个既有顶点：交点落在距该顶点 $\sim 10^{-4}$ uv 处。照实插入会得到近零长边与 sliver 面，随后沿分支排序、弦连接、cut 标记全部退化（同一位置出现 2~3 个相距 3×10^{-5} 的顶点），曲线链断开。解法是把这类交点**吸附到该顶点**（分类为 `Side::OnLine`），几何偏差上界即本容差（ $npts = 1600$ 时约为边长的 1%，肉眼不可见），而拓扑变得严格干净。

容差**必须与面无关**，否则同一条边的两侧面对同一顶点给出不同分类。故取参数域垂直距离，换算到 ϕ 空间时乘 $\sqrt{1+k^2}$ ：

$$\text{snap}(k) = \text{KNOT_SNAP_UV} \cdot \sqrt{1+k^2} \quad (\text{knot_snap_phi})$$

整数 k 下 ϕ 的跨面差恒为 2π 的整数倍，被分支索引吸收，故分类跨面一致。

禁止跨面焊接顶点 COINCIDENT_TOL 只在同一个面内比较（见 `face_vertex_near`）。曾有一版维护全局“接缝焊接表”，按 3D 坐标跨面复用顶点，企图让曲线在接缝处共享顶点——那是错的：会制造 pinch 顶点（一个顶点同属两侧互不相连的扇形），使顶点轨道遍历只覆盖半个扇形、面切不开。跨接缝的连通性只应由 `build_seam_pairs` 在需要时按 3D 距离配对处理。

3 面跨越的分支范围：knot_branch_range

对整张面算出 $\phi \in [\phi_{\min}, \phi_{\max}]$ (`face_phi_values`)，则需枚举的分支为

$$n_{\min} = \left\lfloor \frac{\phi_{\min} - \text{snap}}{2\pi} \right\rfloor, \quad n_{\max} = \left\lfloor \frac{\phi_{\max} + \text{snap}}{2\pi} \right\rfloor.$$

两端都是 $\lfloor \cdot \rfloor$ ，因为分支 n 覆盖 $[2\pi n, 2\pi(n+1))$ ，“包含 ϕ 的分支”就是满足 $2\pi n \leq \phi$ 的最大整数 n 。

旧实现的 ceil bug (已修复) 旧代码写作 $n_{\text{lo}} = \lceil (\phi_{\min} - \text{snap}) / 2\pi \rceil$ 。取 $\phi_{\min} = -12.49$ ： $\lceil -1.988 \rceil = -1$ ，而 $2\pi \cdot (-1) = -6.28$ 根本不覆盖 -12.49 ；正确值是 $\lfloor -1.988 \rfloor = -2$ 。于是 $n_{\text{lo}} = -1 > n_{\text{hi}} = -2$ ，上游 `for branch in n_lo..=n_hi` 静默空循环，该面整个没被切。在 Quad 网格 $+k = [3, 10]$ 之类大幅度值时（大量面 ϕ 为负、需多个分支覆盖）尤其致命：区域数退化为 1，整片一色。

末尾另有 swap 防御：极端数值下 snap 让 $n_{\min}$ 跨过 $n_{\max}$ 时交换两者，杜绝任何静默空循环。

4 相交检测: `intersect_face_knot_branch`

给定面顶点 $\{(u_i, v_i)\}$ 、参数 k 与分支号 `branch`，目标常数 $\text{target} = 2\pi \cdot \text{branch}$ 。

Step 1: 归一化、解缠、算 ϕ

$$\bar{u}_i = \text{normalize_uv}(u_i), \quad a_i = \text{unwrap}(\bar{u}_i, \bar{u}_0), \quad b_i = \text{unwrap}(\bar{v}_i, \bar{v}_0), \quad \phi_i = b_i - k a_i$$

Step 2: 顶点分类（吸附优先）

$$\text{side}_i = \begin{cases} \text{OnLine} & |\phi_i - \text{target}| < \text{snap}(k) \\ \text{Less} & \phi_i - \text{target} < 0 \\ \text{Greater} & \phi_i - \text{target} > 0 \end{cases}$$

若全部顶点均为 `OnLine`（面整体贴合曲线），直接返回全体顶点作为 `on_line_vertices`、无边交点。若既无 `Less` 又无 `Greater` 的横跨、也没有 `OnLine`，返回 `None`。

Step 3: 求边交点（跳过已吸附的边）

对每条边 (i_0, i_1) :

- 任一端点为 `OnLine` $\Rightarrow$ 跳过。该边的穿越由该顶点本身代表，不再插点。旧实现改用 $t < \text{MERGE_EPS}$ 判断，但 t 的阈值与吸附容差无关：吸附后 d_0 最大可达 `snap`，算出的 t 远超 10^{-4} ，会在顶点旁 $\sim \text{snap}$ 处再插一个近重合顶点，正是 `sliver` 的来源。
- 两端同侧 $\Rightarrow$ 跳过。
- 否则线性插值 $t = -d_0 / (d_1 - d_0)$ ， $d_j = \phi_{i_j} - \text{target}$ ，夹到 $[0, 1]$ ；3D 位置按 t 线性插值，新 UV 由解缠坐标插值后经 `clamp_uv` 夹回域内。

5 面切割: `cut_face_knots`

这是与旧文档最本质的结构差异：一个面在一次调用内处理所有曲线的所有分支，把全部切割弦收集后统一落地。

```
1 let fv = get_face_uv(mesh, face);
2 let mut chords: Vec<(VertexId, VertexId)> = Vec::new();
3 for &k in k_values { // 所有曲线
4     let (n_lo, n_hi) = knot_branch_range(&fv, k, uv_range);
5     for branch in n_lo..=n_hi { // 所有分支
6         let Some(isect) = intersect_face_knot_branch(&fv, k, branch, uv_range)
7         else { continue };
8         // 边交点插入 + OnLine 顶点直接复用
9         let mut pts = ...; // insert_boundary_point / VertexId
10        if pts.len() < 2 { continue; }
11        let ordered = order_verts_along_branch(mesh, face, &pts, k, uv_range);
```

```

12     for w in ordered.windows(2) {           // 开口折线，不闭合
13         let (a, b) = (w[0], w[1]);
14         if mark_edge_cut_between(mesh, face, a, b)
15             || mark_edge_cut_anywhere(mesh, a, b) { /* 沿已有边切 */ }
16         else { chords.push((a, b)); }      // 真正的对角线弦
17     }
18 }
19 }
20 for (a, b) in chords { add_chord(mesh, a, b, uv_range, 0); }

```

为何必须一次性处理所有曲线 逐条曲线串行切割会让第二条曲线的弦落在已被第一条切出的子面上，两条曲线在共享顶点处 串接成单条环——那样不会把环面分开，区域会错误合并。正确做法是在同一切面内同时添加各曲线的弦；若两条曲线在面内相交，由 `insert_crossing_vertex` 在交点插入内部顶点，使其成为真正的**横截交叉**（四面对应）。

沿分支方向排序（`order_verts_along_branch`） 分支在参数域内是直线，面内交点必然共线，排序键取其方向 $(1, k)$ 上的投影 $u + kv$ ，排完连成**开口折线**。旧实现用 `order_verts_on_face`（绕面边界排序）并首尾闭合：2 个点时重复连同一条弦， ≥ 3 个点时连成闭合三角形，凭空多出屏障边 $\Rightarrow$ 区域被错误切碎。

三段式落地，任何一段都不许静默丢弃

1. `mark_edge_cut_between(face, a, b)`: 曲线正好走在本面已有边上（角点、接缝、被前一条曲线切出的子边上常见） $\Rightarrow$ 把该无向边（含 twin）标为屏障。
2. `mark_edge_cut_anywhere(a, b)`: a, b 可能被前面的弦切成了相邻子面的公共边 $\Rightarrow$ 在 a 的整个扇形里找。
3. 两者都不中才是真正的对角线弦，交给 `add_chord`: 由 `find_face_with_both` 找到共面则 `split_face`，否则递归调用 `insert_crossing_vertex`（深度上限 64）。

旧实现在第 1、2 步失配时直接丢弃该段，切割链在此断成 degree-1 端点。

边界安全的顶点扇形（`faces_around_vertex`） 单向轨道 $he \rightarrow next(twin(he))$ 一旦遇到 `twin = MAX`（接缝 / 域边界）就断，只覆盖一侧扇形。旧实现在此直接返回，导致落在边界顶点（ $v \equiv 0/2\pi$ 接缝点、四个角点）上的弦找不到共面而被丢弃。现改为正反双向遍历，完整覆盖扇形。

6 区域指派: `assign_connected_components_knot`

解析拓扑不变量

对两条曲线参数 k_1, k_2 （整数， $d = |k_2 - k_1|$ ），定义

$$L(u, v) = \left(\left\lfloor \frac{v - k_1 u}{2\pi} \right\rfloor - \left\lfloor \frac{v - k_2 u}{2\pi} \right\rfloor \right) \bmod d$$

L 在环面粘合下不变:

- $u \rightarrow u + 2\pi$: 两个 $[\cdot]$ 分量各减 k_1 、 k_2 ，差变化 $k_2 - k_1$ ，模 d 后不变；
- $v \rightarrow v + 2\pi$: 两个分量各加 1，差不变。

故 L 是合法的环境拓扑不变量，直接给出每个面所属区域，无需任何 flood-fill。

实现要点

```

1 let d_size = if d < 1e-9 { 0 } else { (d + 0.5) as isize };
2 if d_size == 0 {                                     // 单条曲线 / k1 == k2
3     // (1,k) 是**非分离**闭曲线，不分割环面 => 1 区域
4     for f in &mut mesh.faces { if f.valid {
5         f.component_id = Some(0); f.patch_index = Some((0, 0)); } }
6     return 1;
7 }
8 for each valid face {
9     let (cu_raw, cv_raw) = circular_centroid(mesh, &hes, ...); // 接缝环绕安全
10    let cu = normalize_uv(cu_raw, min_u, max_u); // 归一化到 [0, 2pi)
11    let cv = normalize_uv(cv_raw, min_v, max_v);
12    let n1 = ((cv - k1 * cu) / two_pi).floor() as isize;
13    let n2 = ((cv - k2 * cu) / two_pi).floor() as isize;
14    let raw = (((n1 - n2) % d_size) + d_size) % d_size; // 非负取模
15 }
16 // 最后把出现过的 raw 值重映射为紧凑索引 0..n, 写入 component_id 与 patch_index

```

三个细节缺一不可：

- 面中心用 `circular_centroid`: 先相对首顶点解缠再取均值，跨接缝面的质心才落在面的真实位置；直接算算术平均会把跨缝面的质心甩到域中央。
- 先 `normalize_uv` 到 $[0, 2\pi)$: 曲线方程定义在归一化角度空间，故 UI 自定义的非 $(0, 2\pi)$ `uv_range` 也能正确处理。
- 紧凑重映射: 把实际出现的 L 值映射为连续 $0..n$ ，保证调色板与补片索引不留空洞。

区域数定理

环面上两条 (p, q) 、 (p', q') 闭曲线把环面分成

$$|pq' - p'q|$$

块。本项目的曲线均为 $(1, k)$ 形式，故区域数 $= |1 \cdot k_2 - 1 \cdot k_1| = |k_2 - k_1|$ 。单条 $(1, k)$ 曲线 ($k_1 = k_2$) 是非分离闭曲线：割开后仍连通，1 个区域。

为何取代 flood-fill

旧 `assign_connected_components` 沿 `twin` 半边 flood-fill、以 `cut` 边为屏障，并跨 `twin = MAX` 的边界半边按 `build_seam_pairs` 配对“缝合”以恢复环面拓扑。该路径在大幅 k 值下本质脆弱：曲线在接缝处只切出端点 (split 出顶点)，接缝边自身 `cut = false`，缝合逻辑于是跨缝连通、把本应切开的区域合并回一块。实测 $k = [3, 10]$ (应 7 块) 得 `comps = 1`；补加“端点不是切点则

不缝合”的条件仍不足——总有足够多接缝边被缝合。 $k = [3, 6]$ 之所以曾侥幸正确，只是切点位置的数学巧合。

因此 knot 模式一律走解析公式：`assign_connected_components` (flood-fill) 仅保留给导入的 OBJ 网格——那里没有环面接缝语义可利用。

7 与 Grid 模式的区别

环面内部的 U/V 常值切割线同样是**非分离**曲线，整个网格切完仍只有 1 个拓扑连通块。因此 Grid 模式不能按 `component_id` 着色，而按网格单元：

$$\text{component_id} = p_u \cdot \text{num}_v + p_v$$

两种模式的调色板分支不同：Grid 传 $(\text{num}_u, \text{num}_v)$ 走 2D 分支，Knot 传 $(n, 1)$ 走 1D 分支，入口同为 `generate_patch_colors`。分派在 `app.rs` 的 `reapply_cuts` 与 `recolor_patches` 中按 `cut_mode` 完成，两条链路互不干扰。

1D 调色板与 PBR 1D 分支的色相必须均匀铺开 $h_i = i/n$ ；旧公式 $(j + 4i) \bmod 8$ 在 $n_v = 1$ 时退化 ($h \in \{0, 4, 0\} \Rightarrow$ 两红一青)。此外饱和色 albedo 经 PBR 高光 + ACES 色调映射会被推向白色（各通道趋近 1.0），使各区域在屏幕上无法区分，故 1D 调色板的 `lightness` 上限取 0.40（PBR 加亮后最大通道约 0.48，仍在色相可辨区）。

8 回归测试与诊断工具

测试	断言
<code>test_knot_seam_welding_three_regions</code>	$npts \in \{200, 400, 800, 1600\} \times \text{seed } 1..12$ 共 48 组， $k = [3, 6]$ 恰 3 区域
<code>test_knot_single_curve_one_region</code>	单条 $(1, k)$ 曲线恰 1 区域
<code>test_diag_seam_and_barrier</code>	多组 k 的区域数： $[1, 3] \rightarrow 2$ 、 $[2, 5] \rightarrow 3$ 、 $[3, 6] \rightarrow 3$ 、 $[3, 9] \rightarrow 6$
<code>diag_k310_components_and_patch_palette (#[ignore])</code>	$k = [3, 10]$ 恰 7 区域
<code>test_diag_k36_leak (#[ignore])</code>	用同一解析标签定位泄漏边

质量基线：`cargo test` 全绿、`cargo clippy --all-targets` 零 warning。

9 已知边界与注意事项

- **多分支切割开销**：粗网格 + 大 k 时一个面可能跨多条分支，全部切开会显著增加面数。Grid 链路设有面数硬上限 `MAX_FACES = 150000`；knot 链路无此上限，因为它只单遍扫描初始面集 (`initial_face_count`) 而不迭代，规模天然有界。

- **单顶点接触**：曲线只碰到面的一个顶点时 `pts.len() < 2`，不切分。面整体位于曲线一侧，行为正确。
- **质心 vs 几何**：区域指派用面质心。切割正确时每个面整体落在一个区域内，质心判据与几何一致；若某面本应被切却漏切，其质心落入一侧而部分几何落入另一侧，表现为区域边界“锯齿”——这类症状应回溯到切割链路（尤其 `knot_branch_range` 与弦落地三段式），而非区域公式。
- **k 必须为整数**： L 的不变性依赖 $k_1, k_2 \in \mathbb{Z}$ ($u \rightarrow u + 2\pi$ 时分量恰好整数移位)。UI 的 k 为整数拖动条，满足该前提。